Question 1. Short Answer Questions (60 points)

Fill in the box with the answer.

- Removed in publishable version -

Question 2. Coding 1 (20 points)

Complete the overridden method void `addFirst(T item)`.

It is a method of the class `DelimitedARDeque`, a subclass of an iterable `ARDeque`.

It adds an item to the front of the object of `DelimitedARDeque`,

but it will throw an `IllegalArgumentException` if the item already occurs in it more than once.

Test case:

```
DelimitedARDeque<String> dd = new DelimitedARDeque<>();
dd.addFirst("a");
dd.addFirst("b");
try {
    dd.addFirst("a");
    dd.printDeque();
} catch (IllegalArgumentException e) {
    System.out.println("AddFirst failed");
}                                                    → AddFirst failed
```

Question 3. Coding 2 (20 points)

Complete the method `boolean equals(Object that)` of an iterable `ARDeque` that overrides the one in `Object`.

To be the same, they must be of the same size, and must share at most 1 different items at the same position.

Suppose `[]` is an `ARDeque<Integer>`. Then:

```
[1, 2, 3].equals([1, 2, 5])   →    true
[1, 3, 2].equals([1, 2, 3])   →    false
```

---------- THIS IS THE END OF THE DOCUMENT ----------

CPT204 2022-2023 F S2 - Suggested Answers

Note: Question 1 short-answer section is removed in the publishable paper, so only Q2-Q3 can be answered from the available file.

Question 2 Coding 1 - addFirst(T item)

Idea:
- Before adding, count whether item already occurs more than once in the current deque.
- If it already occurs at least once and adding it would make it occur more than once, throw IllegalArgumentException.
- Otherwise add it to the front.

Typical answer pattern:

```
public void addFirst(T item) {
    int count = 0;
    for (T x : this) {
        if (item == null ? x == null : item.equals(x)) {
            count++;
        }
    }
    if (count >= 1) {
        throw new IllegalArgumentException();
    }
    super.addFirst(item);
}
```

If the wording is interpreted as "already occurs more than once before adding", change the condition to count > 1. The given test case expects the second "a" to fail, so count >= 1 is the useful implementation.

Question 3 Coding 2 - equals(Object that)

Requirement:
- same type / compatible ARDeque
- same size
- at most 1 different item at the same position

Typical answer pattern:

```
@Override
public boolean equals(Object that) {
    if (this == that) return true;
    if (!(that instanceof ARDeque<?>)) return false;

    ARDeque<?> other = (ARDeque<?>) that;
    if (this.size() != other.size()) return false;

    Iterator<T> it1 = this.iterator();
    Iterator<?> it2 = other.iterator();
    int different = 0;

    while (it1.hasNext() && it2.hasNext()) {
        T a = it1.next();
        Object b = it2.next();
        if (!(a == null ? b == null : a.equals(b))) {
            different++;
            if (different > 1) return false;
        }
    }
```

```
    return true;
}
```

Question 1. Short Answer Questions (60 points)

Fill in the box with the answer.

- *Removed in publishable version* –

Question 2. Coding 1 (20 points)

Complete the overridden method void `addLast(T item)`.

It is a method of the class `DelimitedARDeque`, a subclass of an iterable `ARDeque`.

It adds an item at the back of the object of `DelimitedARDeque`,

but it will throw an `IllegalArgumentException` if the item already occurs in it more than once.

Test case:

```
DelimitedARDeque<String> dd = new DelimitedARDeque<>();
dd.addLast("a");
dd.addLast("b");
try {
    dd.addLast("a");
    dd.printDeque();
} catch (IllegalArgumentException e) {
    System.out.println("AddLast failed");
}                                                    → AddLast failed
```

Question 3. Coding 2 (20 points)

Complete the method `boolean equals(Object that)` of an iterable `ARDeque` that overrides the one in `Object`.

To be the same, they must be of the same size, and must share at most 2 different items at the same position.

Suppose `[]` is an `ARDeque<Integer>`. Then:

```
[1, 2, 3, 4].equals([1, 2, 5, 6])   →    true
[1, 2, 3, 4].equals([1, 4, 2, 3])   →    false
```

---------- THIS IS THE END OF THE DOCUMENT ----------

CPT204 2022-2023 R S2 - Suggested Answers

Note: Question 1 short-answer section is removed in the publishable paper, so only Q2-Q3 can be answered from the available file.

Question 2 Coding 1 - addLast(T item)

Idea:
- Before adding, count whether item already occurs in the current deque.
- If adding it would make it occur more than once, throw IllegalArgumentException.
- Otherwise add it to the back.

Typical answer pattern:

```
public void addLast(T item) {
    int count = 0;
    for (T x : this) {
        if (item == null ? x == null : item.equals(x)) {
            count++;
        }
    }
    if (count >= 1) {
        throw new IllegalArgumentException();
    }
    super.addLast(item);
}
```

Question 3 Coding 2 - equals(Object that)

Requirement:
- same type / compatible ARDeque
- same size
- at most 2 different items at the same position

Typical answer pattern:

```
@Override
public boolean equals(Object that) {
    if (this == that) return true;
    if (!(that instanceof ARDeque<?>)) return false;

    ARDeque<?> other = (ARDeque<?>) that;
    if (this.size() != other.size()) return false;

    Iterator<T> it1 = this.iterator();
    Iterator<?> it2 = other.iterator();
    int different = 0;

    while (it1.hasNext() && it2.hasNext()) {
        T a = it1.next();
        Object b = it2.next();
        if (!(a == null ? b == null : a.equals(b))) {
            different++;
            if (different > 2) return false;
        }
    }
    return true;
}
```

CPT204 2024
Final Exam Question

Short Answer Questions

1. A _____ variable stores the actual values of primitive data types directly, while a _____ variable stores the address of an object in memory.

2. _____ is a feature in Java where a subclass provides a specific implementation of a method that is already provided by its parent class, whereas _____ is a feature that allows a class to have more than one method having the same name, but with different parameter lists.

3. The outputs of the following code are _____ and _____.

```
public class Test {
   public static void main(String[] args) {
      Object o1 = new Object();
      Object o2 = new Object();
      System.out.print((o1 == o2) + "and" + (o1.equals(o2)));
   }
}
```

4. An _____ is a class-like construct that contains only constants, abstract methods, default methods, and static methods. In many ways, an interface is similar to an abstract class, but an abstract class can contain _____.

5. The output from the following code is _____.

```
public class Main {
      public static void main(String[] args) {
            ArrayList<String> list = new ArrayList<String>();
            list.add("New York");
            ArrayList<String> list1 = (ArrayList<String>) (list.clone());
            list.add("Atlanta");
            list1.add("Dallas");
            System.out.println(list1);
      }
}
```

6. The program below would encounter a compilation error because the Fruit class does not implement the java.lang. Comparable interface and the Fruit objects are not comparable. This is due to the problem in Line ____.

```
1.public class Test {
2.       public static void main(String[] args) {
3.             Fruit[] fruits = {new Fruit(2), new Fruit(3), new Fruit(1)};
```

```
4.              Arrays.sort(fruits);
5.      }
6.}

class Fruit {
   private double weight;

   public Fruit(double weight) {
      this.weight = weight;
   }
}
```

7. A _____ class or method permits you to specify allowable types of objects that the class or method can work with. If you attempt to use a class or method with an incompatible object, the _____ will detect the error.

8. The method header is left blank in the following code. Fill in a generic method header _____.

```
public class GenericMethodDemo {
   public static void main(String[] args ) {
      Integer[] integers = {1, 2, 3, 4, 5};
      String[] strings = {"London", "Paris", "New York", "Austin"};

      print(integers);
      print(strings);
   }

  public static_____ {
      for (int i = 0; i < list.length; i++)
         System.out.print(list[i] + " ");
      System.out.println();
   }
}
```

9. In Java collections, the _____ is efficient for retrieving elements and for adding and removing elements at the end of the list. On the other hand, a _____ is better suited for adding and removing elements at any position within the list, although locating specific elements for operations is not as efficient.

10. Suppose that list1 is a list that contains the strings red, yellow, and green, and that list2 is another list that contains the strings red, yellow, and blue. After executing list1.remove(list2), list1 contains _____.

11. _____ store objects that are processed in a last-in, first-out fashion. _____

store objects that are processed in a first-in, first-out fashion.

12. The output from the following code is _____.

```
public class Test {
   public static void main(String[] args) {
      PriorityQueue<Integer> queue =
         new PriorityQueue<Integer>(
            Arrays.asList(20, 40, 60, 10, 50, 30));

      while (!queue.isEmpty())
         System.out.print(queue.poll() + " ");
   }
}
```

13. Show the output of the following code_____

```
Set<String> set = new LinkedHashSet<>();
set.add("ABC");
set.add("FGH");
System.out.println(set);
```

14. Complete the following code to create an empty hash set of integers:

```
Set<Integer> set = new _____;
```

15. In a library management system where you need to frequently check the availability and information of books with unique identifiers (e.g., the book ID), the most suitable data structure to store the books in the library would be a _____.

16. The _____ is a theoretical approach for analyzing the performance of an algorithm. It estimates how fast an algorithm's execution time increases as the input size increases, which enables you to compare two algorithms by examining their growth rates.

17. An algorithm with the _____ time complexity is called a linear time algorithm and an algorithm with the _____ time complexity is called a logarithmic algorithm.

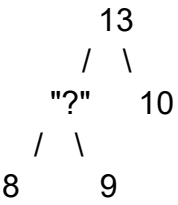
18. Use the Big O notation to estimate the time complexity of the following method: _____

```
public static void mD(int[] m) {
   for (int i = 0; i < m.length; i++) {
      System.out.print(m[i] + " ");
   }
```

19. The worst-case time complexity for selection sort, insertion sort, bubble sort, and quick sort algorithms is _____ .

20. Suppose a list is [12, 9, 15, 4, 20, 11]. After the first pass of bubble sort, the list becomes _____

21. Indicate the number of possible values that can be put in the '?' place to maintain the max heap_____



22. In a graph, if two vertices are connected by two or more edges, these edges are called _____. A_____ is the one in which every two pairs of distinct vertices is connected by an edge.

23. There are two popular ways to traverse a graph, namely _____ and _____ .

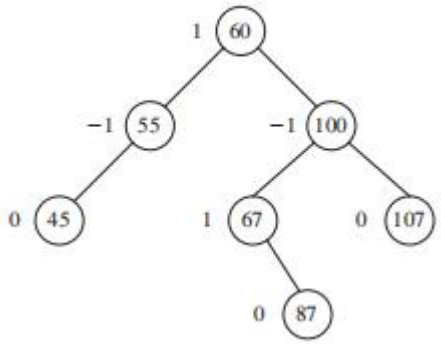
24. In a network of cities connected by roads of varying lengths, if you want to connect all the cities in the most cost-effective way possible, you would use the _____ algorithm. However, if you want to find the shortest route from one specific city to another, you would use the _____ algorithm.

25. The time complexity for inserting and deleting an element into a binary search tree are _____ and _____ respectively, assuming that the tree is balanced.

26. In a binary tree, a _____ traversal visits the root node first, followed by the left subtree, and then the right subtree. Conversely, a _____ traversal visits the left subtree first, followed by the right subtree, and finally the root node..

27. In AVL tree, A node is said to be _____ if its balance factor is -1, and it is said to be _____ if its balance factor is +1.

28. Given the original AVL tree below, after adding element 42, the _____ rotation would be performed to rebalance the tree.



29. _____ and _____ methods are usually taken to deal with hashing collision where two different keys are mapped to the same index in a hash table.

30. In hash tables, _____ probing resolves collisions by sequentially checking the next available slots, while _____ probing resolves collisions by checking slots at increasing squared distances from the original hash.

Coding Questions

Q1. A real estate analyst is comparing the prices of recently sold luxury homes in a prestigious neighborhood. The prices of these homes are as follows: $1,250,000.1, $1,750,000.3, $2,100,000.6, $1,900,000.8, and $2,300,000.2.

Complete the following method that find the maximum element in an ArrayList of generic elements, return null if the ArrayList is empty, and and return the maximum value of the ArrayList that was passed to the function if it is not null.
public static <E extends Comparable<E>> E max(ArrayList<E> list)

Test cases:
ArrayList<Double> prices2 = new ArrayList<>();
System.out.println(maxPrice(prices2)); // -> null

ArrayList<Double> prices1 = new ArrayList<>(Arrays.asList(1250000.1, 1750000.3, 2100000.6, 1900000.8, 2300000.2));
System.out.println(maxPrice(prices1)); // -> 2300000.2

White board (where students code answers):

```
import java.util.ArrayList;
import java.util.Arrays;
```

```
public class RealEstateAnalyzer {

	public static void main(String[] args) {

		// Printing Test case with an empty list

		// Printing Test case with highest prices

	}

	public static <E extends Comparable<E>> E max(ArrayList<E> list) {

		// Empty list case
		return null;


		// Highest price case
		return highestPrice;
	}
}
```

Q2. You are tasked with enhancing a flight boarding system for an airline. The airline categorizes passengers into three classes: "First Class," "Business Class," and "Economy Class." The following three priority queues represent the passengers waiting to board:

First Class Queue: {"Alice", "Bob", "Charlie", "Diana", "Ethan", "Fiona"}
Business Class Queue: {"Charlie", "Diana", "Grace", "Ian"}
Economy Class Queue: {"Alice", "Grace", "Ethan", "Hannah", "Ian"}

You job is to find the exclusive First Class passengers who are only in "First Class" and not in "Business Class" or "Economy Class", by completing the 3 tasks indicated in the following white board.

White board (where students code answers):

```
import java.util.Arrays;
import java.util.HashSet;
import java.util.PriorityQueue;
import java.util.Set;
```

```
public class FlightBoardingSystem {

	public static void main(String[] args) {
		// Task 1: Create PriorityQueues for each class of passengers

		// Task 3: Call findExclusiveFirstClassPassengers() method and print
		out the exclusive First Class passengers

	}


	public static Set<String> findExclusiveFirstClassPassengers(
			PriorityQueue<String> firstClassQueue,
			PriorityQueue<String> businessClassQueue,
			PriorityQueue<String> economyClassQueue) {

		// Task 2: Implement this method to find exclusive First Class
		passengers

		return exclusiveFirstClass;
	}
}
```

CPT204 2023-2024 F S2 - Suggested Answers

Short Answer Questions

1. primitive; reference
2. overriding; overloading
3. false and false
4. interface; concrete methods / instance variables / constructors
5. [New York, Dallas]
6. Line 4
7. generic; compiler
8. <E> void print(E[] list)
9. ArrayList; LinkedList
10. list1.remove(list2) removes the object list2 if present. Since list2 is not an element of list1, list1 remains [red, yellow, green].
11. Stacks; Queues
12. 10 20 30 40 50 60
13. [ABC, FGH]
14. HashSet<Integer>()
15. HashMap
16. Big O notation
17. O(n); O(log n)
18. O(n)
19. O(n^2)
20. [9, 12, 4, 15, 20]
21. 3 possible integer values: 10, 11, 12. If strict uniqueness is required, 11 and 12 only.
22. parallel edges; complete graph
23. DFS; BFS
24. Prim's algorithm; Dijkstra's algorithm
25. O(log n); O(log n)
26. preorder; postorder
27. left-heavy; right-heavy
28. Rotation depends on the figure in the original image. Use AVL insertion rule: LL -> right rotation; RR -> left rotation; LR -> left-right rotation; RL -> right-left rotation.
29. open addressing; separate chaining
30. linear; quadratic

Coding Question 1 - RealEstateAnalyzer

```
import java.util.ArrayList;
import java.util.Arrays;

public class RealEstateAnalyzer {
    public static void main(String[] args) {
        ArrayList<Double> prices2 = new ArrayList<>();
        System.out.println(max(prices2));

        ArrayList<Double> prices1 = new ArrayList<>(Arrays.asList(
                1250000.1, 1750000.3, 2100000.6, 1900000.8, 2300000.2));
        System.out.println(max(prices1));
    }

    public static <E extends Comparable<E>> E max(ArrayList<E> list) {
        if (list == null || list.isEmpty()) {
            return null;
        }
        E highestPrice = list.get(0);
        for (int i = 1; i < list.size(); i++) {
            if (list.get(i).compareTo(highestPrice) > 0) {
                highestPrice = list.get(i);
```

```
            }
        }
        return highestPrice;
    }
}
```

Coding Question 2 - FlightBoardingSystem

```
import java.util.Arrays;
import java.util.HashSet;
import java.util.PriorityQueue;
import java.util.Set;

public class FlightBoardingSystem {
    public static void main(String[] args) {
        PriorityQueue<String> firstClassQueue = new PriorityQueue<>(
                Arrays.asList("Alice", "Bob", "Charlie", "Diana", "Ethan", "Fion
a"));
        PriorityQueue<String> businessClassQueue = new PriorityQueue<>(
                Arrays.asList("Charlie", "Diana", "Grace", "Ian"));
        PriorityQueue<String> economyClassQueue = new PriorityQueue<>(
                Arrays.asList("Alice", "Grace", "Ethan", "Hannah", "Ian"));

        System.out.println(findExclusiveFirstClassPassengers(
                firstClassQueue, businessClassQueue, economyClassQueue));
    }

    public static Set<String> findExclusiveFirstClassPassengers(
            PriorityQueue<String> firstClassQueue,
            PriorityQueue<String> businessClassQueue,
            PriorityQueue<String> economyClassQueue) {
        Set<String> exclusiveFirstClass = new HashSet<>(firstClassQueue);
        exclusiveFirstClass.removeAll(businessClassQueue);
        exclusiveFirstClass.removeAll(economyClassQueue);
        return exclusiveFirstClass;
    }
}
```

Expected exclusive first-class passengers: Bob, Fiona.

CPT204 2024
Resit Exam Question

Short Answer Question (60 marks, 2 for each)

1. In the following code, ‘number’ is a _____ variable as it stores the actual values of primitive data types directly, while ‘message’ is a _____ variable as it stores the address of an object in memory.

```
public class Example {
    public static void main(String[] args) {
        int number = 42;
        String message = "Hello, world!";
    }
}
```

2. A _____ is a class from which other classes are derived, while a _____ is a class that is derived from another class.

3. In Java, _____ equality checks if two variables refer to the exact same object in memory, while _____ equality checks if two objects have the same state or content, as defined by the equals() method.

4. An _____ can have instance methods that implement a default behavior, while an _____ cannot have instance methods.

5. For scenarios requiring efficient random access through an index without frequent insertion or removal of elements except at the end of the list, the _____ class is more suitable. In contrast, for applications that frequently insert or delete elements at the beginning of the list, the _____ class is a better choice.

6. In the program below, a compilation error occurs at line 4 because the Fruit class does not implement the _____ interface, which is required for the Arrays.sort() method to sort the array of Fruit objects.

```
1.public class Test {
2.      public static void main(String[] args) {
3.          Fruit[] fruits = {new Fruit(2), new Fruit(3), new Fruit(1)};
4.          Arrays.sort(fruits);
5.      }
6.}

class Fruit {
   private double weight;
```

```
    public Fruit(double weight) {
        this.weight = weight;
    }
}
```

7. The following statement would encounter a compilation error because a generic type must be a _____ type. One way to correct the statement is to change the type parameter “int” to _____.

```
ArrayList<int> intList = new ArrayList<>();
```

8. The method header is left blank in the following code. Fill in the header using a generic method _____.

```
public class GenericMethodDemo {
    public static void main(String[] args ) {
    Integer[] numbers = {10, 20, 30, 40, 50};
    String[] cities = {"Tokyo", "London", "Paris", "New York"};
        print(numbers);
        print(cities);
    }

  public static_____ {
        for (int i = 0; i < list.length; i++)
            System.out.print(list[i] + " ");
        System.out.println();
    }
}
```

9. In the code below, ‘LinkedList’ is a _____ without specifying any type parameter, which implies it can hold elements of any type.

```
public class Example {
        public static void main(String[] args) {
                LinkedList list = new LinkedList();
                list.add("World");
                list.add(24);
                list.add(2.71);
        }
}
```

10. A list collection organizes elements in a _____ order, and permits specifying the position where an element is stored. Additionally, elements can be accessed by_____.

11. The _____ interface defines the natural ordering of objects of each class that implements it, whereas the _____ interface defines a separate class to compare objects of another class.

12. The _____ method in the Queue interface retrieves and removes the head of this queue, or null if this queue is empty. The _____ method in the Queue interface retrieves and removes the head of this queue and throws an exception if this queue is empty.

13. Suppose list list1 is [1, 3, 5] and list list2 is [2, 4, 6]. After list1.addAll(list2), list1 is _____.

14. In the following code, the key "101" will correspond to the value "_____" after all the operations.

```
public class Test {
   public static void main(String[] args) {
      Map<String, String> map = new HashMap<>();
      map.put("101", "Alice");
      map.put("102", "Bob");
      map.put("101", "Charlie");
   }
}
```

15. The output of the following code is _____.

```
HashMap<String, Integer> map = new HashMap<>();
map.put("One", 1);
map.put("Two", 2);
map.put("Three", 3);
System.out.println(map.get("Two"));
```

16. In the selection sort algorithm, the _____ element in the array is identified and exchanged with the _____ element in each iteration.

17. An algorithm with the O(n) time complexity is called a_____ algorithm and an algorithm with the O(log n) time complexity is called a _____ algorithm.

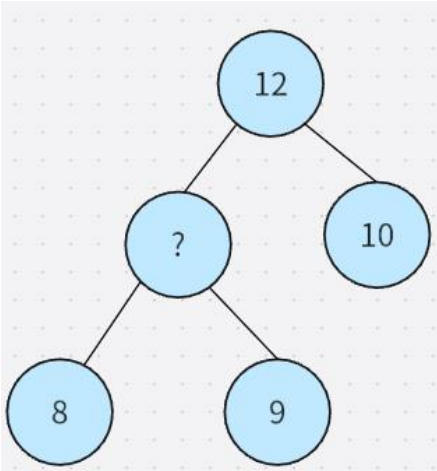
18. _____ is a method for solving complex problems by breaking them down into simpler subproblems where these subproblems overlap, unlike in _____ method where subproblems are more independent.

19. Use the Big O notation to estimate the time complexity of the following method: _____.

```
public static void mD(int[] m) {
   for (int i = 0; i < m.length; i++) {
      for (int j = 0; j < m.length; j++) {
         System.out.print(m[i] + " ");
      }
   }
}
```

20. Suppose a list is [12, 9, 15, 4]. After the second pass of bubble sort, the list becomes _____.

21. What value should best replace the '?' to maintain the max heap strictly?



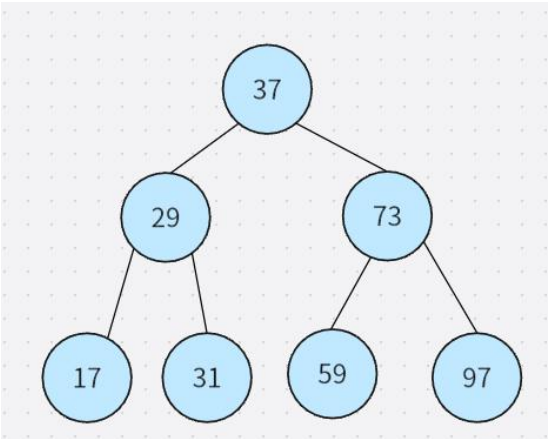
22. A _____ graph is a simple undirected graph in which every pair of distinct vertices is connected by a unique edge, while in a _____ graph, not every pair of vertices is connected by an edge.

23. _____ is a graph traversal method that visits all the vertices of a graph as deeply as possible before backtracking, while _____ visits all the vertices of a graph level by level.

24. _____ is an algorithm in graph theory where the goal is to connect all vertices with the least total weight, while _____ is an algorithm where the goal is to find the path with the least total weight between two vertices.

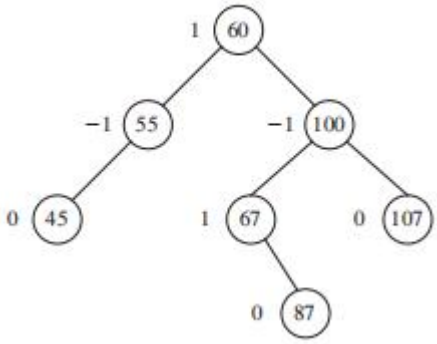
25. In a binary tree, the _____ traversal visits the root node before its children, while the _____ traversal visits the left subtree, then the root, and finally the right subtree.

26. The postorder traversal and the breadth-first traversal of the following tree are _____ and _____ respectively.



27. In an AVL tree, a _____ imbalance occurs when a node is inserted into the right subtree of the right child of A, while a _____ imbalance occurs when a node is inserted into the left subtree of the left child of A.

28. Given the original AVL tree below, after adding element 81, the _____rotation would be performed to rebalance the tree.



29. Collisions occur when two different keys produce the same _____. Besides open addressing, _____, where each hash table slot maintains a linked list of elements mapping to the same slot, is the other way to solve collisions.

30. _____ hashing uses a secondary hash function h' on the keys to determine the increments, aiming to avoid the _____ problem.

Coding Questions (40 marks, 20 for each)

Q1. You are developing a library management system that categorizes books into three sections: "Fiction," "Historical," and "Reference." Each book is placed in a queue according to its section for shelving.

Given three priority queues representing the books waiting to be shelved:

- Fiction Queue: {"The Great Gatsby", "To Kill a Mockingbird", "1984", "Pride

and Prejudice", "Harry Potter", "The Hobbit"}

- Historical Queue: {"Sapiens", "Pride and Prejudice", "A Brief History of Time", "Educated"}
- Reference Queue: {"The Great Gatsby", "A Brief History of Time", "Harry Potter", "Encyclopedia Britannica", "The Hobbit"}

You job is to find books that are present in all three sections (i.e. popular books). Complete the 3 tasks indicated in the following whiteboard.

Whiteboard (where students code answers):

```
import java.util.Arrays;
import java.util.HashSet;
import java.util.PriorityQueue;
import java.util.Set;

public class LibraryManagementSystem {

	public static void main(String[] args) {
		// Task 1: Create PriorityQueues for each section of books

		// Task 3: Call findPopularBooks() method and print out the name of the popular books
	}

	public static Set<String> findPopularBooks(
			PriorityQueue<String> fictionQueue,
			PriorityQueue<String> historicalQueue,
			PriorityQueue<String> referenceQueue) {

		// Task 2: Implement this method to find books that are present in all three sections

		return popularBooks;
	}
}
```

Q2. A human resource system needs to sort employee records based on different attributes. One common requirement is to sort employees by their ID numbers and another is to sort them by their names in a case-insensitive manner.

Given the following array of employee IDs and names:
Integer[] employeeIds = {102, 101, 104, 103, 105};
String[] employeeNames = {"John", "Alice", "bob", "Diana", "Charlie"};

You are required to use insertion sort algorithm to do the following 2 tasks as indicated in the white board.
White board (where students code answers):

```
import java.util.Comparator;

public class EmployeeRecordSorting {

    // Task 1: Use the following insertionSort() method with the Comparable
       interface to sort the array of employee IDs in ascending order.

       public static <E extends Comparable<E>> void insertionSort(E[] list) {

    }

    // Task 2: Use the following insertionSort() method with the Comparator
       interface to sort the array of employee names in a case-insensitive
       manner.

       public static <E> void insertionSort(E[] list, Comparator<? super E>
       comparator) {

       }

  public static void main(String[] args) {

          Integer[] employeeIds = {102, 101, 104, 103, 105};
          String[] employeeNames = {"John", "Alice", "bob", "Diana", "Charlie"};

          insertionSort(employeeIds);
          System.out.println("Sorted Employee IDs:");
          System.out.println(Arrays.toString(employeeIds));

          insertionSort(employeeNames,
String.CASE_INSENSITIVE_ORDER);
          System.out.println("Sorted Employee Names (Case-Insensitive):");
          System.out.println(Arrays.toString(employeeNames));
    }
```

```
}
```

CPT204 2023-2024 R S2 - Suggested Answers

Short Answer Questions

1. primitive; reference
2. superclass; subclass
3. reference; content / logical
4. abstract class; interface
5. ArrayList; LinkedList
6. Comparable
7. reference; Integer
8. <E> void print(E[] list)
9. raw type
10. sequential / linear; index
11. Comparable; Comparator
12. poll(); remove()
13. [1, 3, 5, 2, 4, 6]
14. Charlie
15. 2
16. smallest; first unsorted
17. linear; logarithmic
18. dynamic programming; divide-and-conquer
19. O(n^2)
20. [9, 4, 12, 15]
21. The value depends on the visible heap figure. Choose a value smaller than its parent and larger than its children to preserve max-heap order.
22. complete; incomplete
23. DFS; BFS
24. Prim's algorithm; Dijkstra's algorithm
25. preorder; inorder
26. Depends on the tree figure in the original image. Postorder = left, right, root. Breadth-first = level order.
27. RR; LL
28. Rotation depends on the visible AVL figure after inserting 81.
29. index / hash value; separate chaining
30. double; clustering

Coding Question 1 - LibraryManagementSystem

```
import java.util.Arrays;
import java.util.HashSet;
import java.util.PriorityQueue;
import java.util.Set;

public class LibraryManagementSystem {
    public static void main(String[] args) {
        PriorityQueue<String> fictionQueue = new PriorityQueue<>(
                Arrays.asList("The Great Gatsby", "To Kill a Mockingbird", "1984
",
                        "Pride and Prejudice", "Harry Potter", "The Hobbit"));
        PriorityQueue<String> historicalQueue = new PriorityQueue<>(
                Arrays.asList("Sapiens", "Pride and Prejudice",
                        "A Brief History of Time", "Educated"));
        PriorityQueue<String> referenceQueue = new PriorityQueue<>(
                Arrays.asList("The Great Gatsby", "A Brief History of Time",
                        "Harry Potter", "Encyclopedia Britannica", "The Hobbit")
);

        System.out.println(findPopularBooks(fictionQueue, historicalQueue, refer
enceQueue));
    }
```

```
    public static Set<String> findPopularBooks(
            PriorityQueue<String> fictionQueue,
            PriorityQueue<String> historicalQueue,
            PriorityQueue<String> referenceQueue) {
        Set<String> popularBooks = new HashSet<>(fictionQueue);
        popularBooks.retainAll(historicalQueue);
        popularBooks.retainAll(referenceQueue);
        return popularBooks;
    }
}
```

For the given data, no book appears in all three queues, so the result is [] / empty set.

Coding Question 2 - EmployeeRecordSorting

```
import java.util.Arrays;
import java.util.Comparator;

public class EmployeeRecordSorting {
    public static <E extends Comparable<E>> void insertionSort(E[] list) {
        for (int i = 1; i < list.length; i++) {
            E current = list[i];
            int k;
            for (k = i - 1; k >= 0 && list[k].compareTo(current) > 0; k--) {
                list[k + 1] = list[k];
            }
            list[k + 1] = current;
        }
    }

    public static <E> void insertionSort(E[] list, Comparator<? super E> comparator) {
        for (int i = 1; i < list.length; i++) {
            E current = list[i];
            int k;
            for (k = i - 1; k >= 0 && comparator.compare(list[k], current) > 0; k--) {
                list[k + 1] = list[k];
            }
            list[k + 1] = current;
        }
    }

    public static void main(String[] args) {
        Integer[] employeeIds = {102, 101, 104, 103, 105};
        String[] employeeNames = {"John", "Alice", "bob", "Diana", "Charlie"};

        insertionSort(employeeIds);
        System.out.println(Arrays.toString(employeeIds));

        insertionSort(employeeNames, String.CASE_INSENSITIVE_ORDER);
        System.out.println(Arrays.toString(employeeNames));
    }
}
```

Expected sorted IDs: [101, 102, 103, 104, 105]
Expected sorted names: [Alice, bob, Charlie, Diana, John]

CPT204 Final Exam Sample Questions - AY2425

Part A. Gap-fill Questions

1. _____ allows different classes to define methods with the same name but behave differently, while _____ ensures that internal class data is not directly accessible from outside the class.

2. _____ store objects that are processed in a last-in, first-out fashion. _____ store objects that are processed in a first-in, first-out fashion.

3. Given the following method call:

```
public static void update(int num, int[] array) {
    num = 100;
    array[0] = 50;
    }
```

What is the printed value of x in the main method?

```
int x = 10;
int[] arr = new int[1];
update(x, arr);
System.out.println("x = " + x);
```

4. What is the default value of myArray[2] after this declaration?

```
boolean[] myArray = new boolean[5];
```

5. What is the output of the following code:

```
String s1 = "Hello";
String s2 = new String("Hello");
```

System.out.println(s1 == s2);

6. Given the Student class below:

```
public class Student {
        private List<String> courses;

        public List<String> getCourses() {
            return courses;
        }
}
```

Although courses is private, the class is still not immutable because the getter returns a/an _____ to a mutable object.

7. public boolean equals(Object o) {

```
if (o instanceof Circle) {
return radius == ((Circle) o).radius;
   }
   return false;
}
```

After _____, you can access the field. Equality must be based on comparing relevant fields.

8. class B {

```
   public void p(double i){
       System.out.println(i *2);
   }
}
class A extends B   {
```

```
    public void p(int i) {
        System.out.println(i);
    }
}
```

Given that method overloading is based on the reference type at compile time, what is printed by:

```
B a = new A();
a.p(10);
```

9. In the code below:

```
interface T1 {
int K = 1;
}
System.out.println(T1.K);
```

What kind of static member — which is fixed and cannot be changed — is accessed using the interface name T1?

10. House house1 = new House(1, 1750.50);

```
House house2 = (House)(house1.clone());
System.out.println(house1.getWhenBuilt() == house2.getWhenBuilt());
```

The code above performs a/an _____ if clone() is not overridden.

11. If the key is the first element in the array, the linear search completes in _____ comparisons.

12. In our class, we introduced various methods for recording OOP learning, among which a _____ is a means of recording ideas, personal thoughts and experiences, as well as insights you gain in the learning process.

13. We have learned several different data structures in Java. Suppose you are asked to design a container that can support efficient and random access through an index, and efficient insertion and deletion from the end. The best data structure for the container is _____.

14. We have learned several different data structures in Java. Suppose you are asked to design a container that can support efficient and random access through an index; in addition, it supports insertion and deletion from the end by multiple threads concurrently. The best data structure for the container is _____.

15. What is the output of the following code segment

```
PriorityQueue<String> queue = new PriorityQueue<String>(
Arrays.asList("Liverpool", "Suzhou", "London", "Beijing"));
while (!queue.isEmpty())
System.out.print(queue.poll() + " ");
```

16.

Suppose we are given a linked list CPTList and list node CPTListNode outlined below.

```
public class CPTListNode {
    int data;
    CPTListNode next;
}

public class CPTList {
    private CPTListNode front;
    // some methods
}
```

A method called "method1" for the CPTList class is shown below. Assume there are two CPTList objects, list1 and list2. list1 contains 3 nodes of value 1, 2, and 3 (starting from the front). list2 contains another 3 nodes of value 4, 5, and 6 (starting from the front). What would be the

nodes in list1 after list1.method1(list2)? Place a comma between neighboring nodes in your answer, e.g. 1,3,2,4 (although comma is not a part of the linked list data structure).

```
public void method1(CPTList other){
  CPTListNode l = other.front;
  while (l.next != null){
      l = l.next;
    }
   l.next = front;
    front = other.front;
}
```

17. A Map is a special type of Java container that stores a collection of key/value pairs. We have learned 3 types of built-in maps. They are: ____, ____, and TreeMap.

18. Suppose we have 3 containers below each containing the same set of students IDs at xjtlu. container1 is a LinkedHashSet.

container2 is a LinkedList.

container3 is an ArrayList.

The student IDs are stored in a random order in all 3 containers. Now We want to check if a given id, say 2209999, exists in the containers using the method "contains()". The code segment would be like

container1.contains(2209999)

container2.contains(2209999)

container3.contains(2209999)

Which container is likely to have the minimum time consumption?

19. In Big O notation, the term _____ is used to describe algorithms with complexity that grow at a linear rate with the input size.

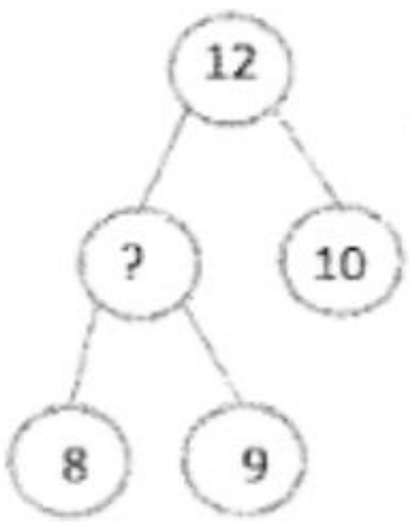
20. The worst-case time complexity for selection sort, insertion sort, bubble sort, and quick sort algorithms is _____.

21.
```
int count = 0;
for (int i = 1; i < n; i *= 2) {
    count++;
}
```

What is the time complexity of the code above?

22. In Merge Sort, the array is repeatedly divided into smaller sublists until each sublist contains only _____ element.

23. What value should best replace the '?' to maintain the max heap strictly?



24. The time complexity for inserting an element into, and deleting an element from, a binary search tree are _____ and _____ respectively, assuming that the tree is balanced.

25. What does Dijkstra's algorithm compute?

26. In an AVL tree, a _____ imbalance occurs when a node is inserted into the right subtree of the right child of A, while a _____ imbalance occurs when a node is inserted into the left subtree of the left child of A.

27. The postorder traversal of the elements in an AVL tree after inserting 15, 10, 20, 25, 8, 12 in this order is _____.

28. The following code uses _____ probing to resolve collisions in a hash table.

```
int index = key % table.length;
while (table[index] != null) {
    index = (index + 1) % table.length; // linear probing
}
table[index] = key;
```

29. What EDI principle is demonstrated by using accessible color schemes in a Java Swing GUI, ensuring that users with visual impairments are not excluded?

30. What EDI principle is reflected when a Java app project is designed to accommodate both younger and older users, screen reader users, and speakers of different languages?

Part B. Coding Questions

Question 1

Scorable Quiz System

Given the following interface:

```
public interface Scorable {
    int getScore();
}
```

Complete the class QuizResult that implements Scorable. The getScore() method should return the percentage score as an integer (e.g., 80 for 8/10). If there are no questions, simply return 0. Use standard rounding (i.e., round to the nearest whole number).

Starter Code:

```
public class QuizResult implements Scorable {
    private int correctAnswers;
    private int totalQuestions;

    public QuizResult(int correctAnswers, int totalQuestions) {
        // TO DO
    }
    // TO DO
}
```

Question 2

You are writing a Java program that implements an airplane ticket. The ticket can be compared by passenger name, ticket price, arrival time, and seat class in that order.

Moreover, the seat class is implemented using an enumeration with First Class, Business, and Economy, compared in that order.

Your task is to complete the TicketComparator class.

Date and Comparator have been imported. You are not allowed to import other classes.

The skeleton code is given below:

```
import java.util.Date;
import java.util.Comparator;

public class AirplaneTicket {
    private String passengerName;
    private int ticketPrice;
    private String flightNumber;
    private Date arrivalTime;
```

```
    private SeatClass seatClass;

    public enum SeatClass {
        FIRST_CLASS, BUSINESS, ECONOMY
    }

    public AirplaneTicket(String passengerName, int ticketPrice, String flightNumber, Date
arrivalTime, SeatClass seatClass) {
        this.passengerName = passengerName;
        this.ticketPrice = ticketPrice;
        this.flightNumber = flightNumber;
        this.arrivalTime = arrivalTime;
        this.seatClass = seatClass;
    }

    public String getPassengerName() {
        return passengerName;
    }
    public int getTicketPrice() {
        return ticketPrice;
    }
    public String getFlightNumber() {
        return flightNumber;
    }
    public Date getArrivalTime() {
        return arrivalTime;
    }
    public SeatClass getSeatClass() {
```

```
        return seatClass;
    }
    public static Comparator<AirplaneTicket> getTicketComparator() {
        return new TicketComparator();
    }
    // Complete the class TicketComparator
    private static class TicketComparator implements Comparator<AirplaneTicket> {

    }
}
}
```

---------- END OF THE CPT 204 FINAL EXAM SAMPLE QUESTIONS ----------

CPT204 2024-2025 F S2 - Searchable Transcript and Suggested Answers

This file provides a clean searchable transcript/answer page for the scanned pap
er. The original image pages are preserved in the combined PDF.

Part A Gap-fill Questions - Suggested Answers

1. Polymorphism; Encapsulation
2. Stacks; Queues
3. x = 10
4. false
5. false
6. reference
7. casting / downcasting
8. 20.0
9. constant / public static final constant
10. shallow copy
11. 1
12. reflective journal / reflective journal-log
13. ArrayList
14. Vector
15. Beijing Liverpool London Suzhou
16. 4,5,6,1,2,3
17. HashMap, LinkedHashMap, TreeMap
18. container1 / LinkedHashSet
19. O(n)
20. O(n^2)
21. O(log n)
22. one
23. 11
24. O(log n); O(log n)
25. Dijkstra's algorithm computes shortest paths from a source vertex to other v
ertices.
26. right-right; left-left
27. 8,12,10,25,20,15
28. linear
29. Inclusion
30. Diversity, or Inclusion if the expected wording focuses on not excluding all
 user groups.

Part B Question 1 - Scorable Quiz System

```
public class QuizResult implements Scorable {
    private int correctAnswers;
    private int totalQuestions;

    public QuizResult(int correctAnswers, int totalQuestions) {
        this.correctAnswers = correctAnswers;
        this.totalQuestions = totalQuestions;
    }

    @Override
    public int getScore() {
        if (totalQuestions == 0) {
            return 0;
        }
        return (int) Math.round((double) correctAnswers / totalQuestions * 100);
    }
}
```

Part B Question 2 - AirplaneTicket Comparator

```
private static class TicketComparator implements Comparator<AirplaneTicket> {
    @Override
    public int compare(AirplaneTicket t1, AirplaneTicket t2) {
        int result = t1.getPassengerName().compareTo(t2.getPassengerName());
        if (result != 0) return result;

        result = Integer.compare(t1.getTicketPrice(), t2.getTicketPrice());
        if (result != 0) return result;

        result = t1.getArrivalTime().compareTo(t2.getArrivalTime());
        if (result != 0) return result;

        return t1.getSeatClass().compareTo(t2.getSeatClass());
    }
}
```

Raw OCR Search Text

CPT204 Final Exam Sample Questions - AY2425
Part A. Gap-fill Questions
1.
differently, while
the class.
_ allows different ciasses to define methods with the same name but behave
ensures that internal class data is not directly accessible from outside
2..
store objects that are processed in a last-in, first-out fashion.
are processed in a first-in, first-out fashion.
Only by 23
3. Given the following method call:
espect Copyrig]
public static void update(int num, intl] array) (
num = 100;
array[0] = 50;
XJTLU
}
Jse Respon
What is the printed value of x in the main method?
Use Only
int x = 10;
int[] arr = new int[1];
update(x, arr);
tern
System.out.printin('x=
store objects that
4. What is the default value of myArray(2] after
this
booleanji myArray = new booiean[5];
tuy
Use Res
5. What is the output of the following code:
String s1 = "Hello";
String s2 = new String("Hello");

opyright
System.out.printin(s1 == s2);
Use
Respe
XJTLU Acar
Jeo eospebic dius sudoent
6, Given the Student class below:
private List‹String› courses;
public List<String> getCourses() {
return courses;
}
Only by 2361387
ect Copyright
Although courses is private, the class is still not immutable because the getter returns a/an
to a mutabie object.
7. public boolean equals(Object o) {
if (o instanceof Circie) (
return radius == ((Circle) o).radius;
}
return false;
}
After
ternal Use
Only by 2361387
ict Copyright
_, you can access the field. Equality must be based on comparing relevant fields.
8. class B (
public void p(double i)X
System.out.printin(i *2);
XJTLU Academic
Use Responsibly & F
}
{
}
class A extends B

===== PAGE 3: page-03.png =====
by 2361387
public void p(int i) {
opyright
System.out.printin(i);
XJTLU AS
Use R& Given that method overloading is based on the reference type at compile time, what is printed
B a = new A();
a.p(10);
9. In the code below:
interface T1 (
int K = 1;
}
emic
Only by 2361387
Respecí Copyright
System.out.printin(T1.K);
What kind of static member -which is fixed and cannot be changed – is accessed using the interface name T1?
10. House house1 = new House(1, 1750.50);
System.out.printinhouse1.getWhenBuiltf)==house2.getWhenBuilti); Only bi 2361387
The code above performs a/an
if cione() is not overridden.

& Respect Copyright
ısibly
11. If the key is the first element in the array, the linear search completes in comparisons.
12. In our ciass, we introduced various methods for recording OOP learning, amon g which a
is a means of recording ideas, personal thoughts and experiences, as well as ins ights
you gain in the learning process.

===== PAGE 4: page-04.png =====
2361387
opyright
13. We have learned several different data structures in Java. Suppose you are a sked to design
a container that can support efficient and random access through an index, and e fficient
insertion and deletion from the end. The best data structure for the container i s
Responsi
14. We have learned several different data structures in Java. Suppose you are a sked to design
a container that can support efficient and random access through an index; in ad dition, it
supports insertion and deletion from the end by multiple threads concurrently. T he best data
structure for the container is
361381
15. What is the output of the following code segment
PriorityQueue<String> queue = new PriorityQueue<String>(
Arrays.asList("Liverpool", "Suzhou", "London", "Beijing"
while (Iqueue.isEmpty())
Only
System.out.print(queue.poll)+"");
U5°
16.
Suppose we are given a linked list CPTList and list node CPTListNode outlined be low
public class CPTListNode (
int data;
CPTListNode next;
tern
}
public class CPTList {
private CPTListNode front;
// some methods
XJTLU Academic Use Only by 236131
Use Responsibly & Respect Copyright
}
A method called "method1" for the CPTList class is shown below. Assume there are two CPTList
objects, list1 and list2. list1 contains 3 nodes of value 1, 2, and 3 (starting from the front). list2
contains another 3 nodes of value 4, 5, and 6 (starting from the front). What wo uld be the

===== PAGE 5: page-05.png =====
answer,cce.g. 1,3,2,4 (although comma is not a part of the linked list data stru cture).
public void method1(CPTList other)(
Use Respor
while (I.next |= nullk

1= I.next;
}
i.next = front;
front = other.front;
}

17. A Map is a special type of Java container that stores a collection of key/value pairs. We
have learned 3 types of built-in maps. They are:
,and TreeMap.
18. Suppose we have 3 containers below each containing the same set of students IDs at xjtlu.
container1 is a LinkedHashSet.
container2 is a LinkedList.
container3 is an ArrayList.

The student IDs are stored in a random order in all 3 containers. Now We want to check if a
given id, say 2209999, exists in the containers using the method "contains()". The code segment
would be llke
container1.contains(2209999)

container2.contains(2209999)
container3.contains(2209999)

lesponsibly &
Which container is likely to have the minimum time consumption?
19. In Big O notation, the term
at a linear rate with the input size.
. is used to describe algorithms with complexity that grow

20. The worst-case time complexity for selection sort, insertion sort, bubble sort, and quick
sort algorithms is

for (inti = 1;i<n;i*= 2) {
count++;
}
What is the time complexity of the code above?
contains only....
element.

23. What value should best replace the '?' to maintain the max heap strictly?

10

24. The time complexity for inserting an element into, and deleting an element from, a binary
search tree are
and
_ respectively, assuming that the tree is balanced.

25. What does Dijkstra's algorithm compute?
XJTLU Acad
Use Responsib
26. In an AVL tree, a
the right child of A, while a
subtree of the left child of A.
_imbalance occurs when a node is inserted into the right subtree of
imbalance occurs when a node is inserted into the left

===== PAGE 7: page-07.png =====
2361387
opyright
27. The postorder traversal of the elements in an AVL tree after inserting 15, 1
0, 20, 25, 8, 12
in this order is
28. The following code uses
XJTLU
Resint index =key % table.length;
while (table[index] != null) {
index = (index + 1) % table.length; // linear probing
probing to resolve collisions in a hash table.
}
table[index] = key;
lly by 2361387
Copyright
29. What EDI principle is demonstrated by using accessible color schemes in a Ja
va Swing GUI,
ensuring that users with visuai impairments are not exciuded?
30. What EDI principle is reflected when a Java app project is designed to accom
modate both
younger and older users, screen reader users, and speakers of different language
s?
Part B. Coding Questions
Question 1
Scorable Quiz System
nter
Given the following interface:
public interface Scorable (
361387
int getScore();
}
J5°
Respect
XJTLU Academic
Use Responsibly &
Complete the class QuizResult that implements Scorable. The getScore() method sh
ould return the
percentage score as an integer (e.g., 80 for 8/10). If there are no questions, s
imply return 0. Use
standard rounding (i.e., round to the nearest whole number).

===== PAGE 8: page-08.png =====
by 2361387
Starter Code:
Copyright
public class QuizResult implements Scorable (
Use Responsib,
private int totalQuestions;
public QuizResult(int correctAnswers, int totalQuestions) {
// TO DO
}
// TO DO

Only by 2361387
lespect Copyright
Question 2
Only
You are writing a Java program that implements an airplane ticket. The ticket ca n be compared
by passenger name, ticket price, arrival time, and seat elass in that order.
Moreover, the seat class is implemented using an enumeration with First Class, B usiness, and
Economy, compared in that order.
Your task is to complete the TicketComparator class.
2361387
Date and Comparator have been imported. You are not allowed to import other clas ses.
The skeleton code is given below:
```
import java.util.Date)
import java.util.Comparator;
XJTLU Academic Use
Use Responsibly & Respeci
public class AirplaneTicket (
private String passengerName;
private int ticketPrice;
private String flightNumber;
private Date arrivalTime;
```

===== PAGE 9: page-09.png =====

```
2361381
righ
private SeatClass seatClass;
public enum SeatClass {
FIRST_CLASS, BUSINESS, ECONOMY
XJTLU
Use Resp
public AirplaneTicket(String passengerName, int ticketPrice, String flightNumber , Date
arrivalTime, SeatClass seatClass) {
this.passengerName = passengerName: 36
this.ticketPrice = ticketPrice;
Only bys
this flightNumber =flightNumbersectCopyright
this.arrivalTime = arrivalfime;
this.seatClass = seatClass;
public String getPassengerName() {
al Use Only
return passengerName;
}
public int getticketpricel n2
return ticketPrice;
}
public String getFlightNumber() (
return flightNumber;
}
public Date getArrivalTime() {
return arrivalTime;
XJTLU Academic Use Only by 2361387
Use Responsibly & Respect Copyright
}
public SeatClass getSeatClass() {
```

===== PAGE 10: page-10.png =====

```
by 2361387
pyright
```

```
return seatClass;
1S°
XJTLU Academic
Respe
public static Comparator<AirplaneTicket> getTicketComparator() {
Use Responsibl
return new TicketComparator();
}
// Complete the class TicketComparator
private static class TicketComparator implements Comparator<AirplaneTicket> {
}
}
204 FINAL
EX
AM
Only
Internal USen
XJTLU Academic Use Only by 2361387
Use Responsibly & Respect Cópyright
```

CPT204 Mock Exam - Question 13
OnlineCourse

Given the following interface:

```
interface Payable {
    double getPaymentAmount();
}
```

Complete the class OnlineCourse that implements Payable.

An online course has:

- courseName
- baseFee
- level
- hasCertificate

The course level surcharge rules are:

- BEGINNER: +0
- INTERMEDIATE: +100
- ADVANCED: +200

If hasCertificate is true, add an extra 50 after adding the level surcharge.

If baseFee is negative, getPaymentAmount() should return 0.

Answer:

```
interface Payable {
    double getPaymentAmount();
}

class OnlineCourse implements Payable {
    private String courseName;
    private double baseFee;
    private Level level;
    private boolean hasCertificate;

    public enum Level {
        BEGINNER, INTERMEDIATE, ADVANCED
    }

    public OnlineCourse(String courseName, double baseFee,
                        Level level, boolean hasCertificate) {
        // TODO: Please complete this method.
    }

    @Override
    public double getPaymentAmount() {
        // TODO: Please complete this method.
    }
}
```

CPT204 Mock Exam - Question 14
KeywordCounter

Given the following interface:

```
interface Countable {
    int getKeywordCount();
}
```

Complete the class KeywordCounter that implements Countable.

A KeywordCounter has:

```
private String text;
private HashSet<String> keywords;
private HashMap<String, Integer> wordCounts;
```

The class counts how many times selected keywords appear in a piece of text.

The input text will contain lowercase words separated by single spaces only. There will be no punctuation.

Rules:

- The constructor should initialise all properties.
- Store all given keywords in the HashSet.
- Count all words in the text using the HashMap.
- getKeywordCount() should return the total number of times any keyword appears in the text.
- If text is null or empty, return 0.

Answer:

```
import java.util.*;

interface Countable {
    int getKeywordCount();
}

class KeywordCounter implements Countable {
    private String text;
    private HashSet<String> keywords;
    private HashMap<String, Integer> wordCounts;

    public KeywordCounter(String text, Collection<String> keywords) {
        // TODO
    }

    private void countWords() {
        // TODO
    }

    @Override
    public int getKeywordCount() {
        // TODO
    }
}
```

CPT204 Mock Exam Questions 13-14 - Completed Suggested Answers

Question 13 - OnlineCourse

```
interface Payable {
    double getPaymentAmount();
}

class OnlineCourse implements Payable {
    private String courseName;
    private double baseFee;
    private Level level;
    private boolean hasCertificate;

    public enum Level {
        BEGINNER, INTERMEDIATE, ADVANCED
    }

    public OnlineCourse(String courseName, double baseFee,
                        Level level, boolean hasCertificate) {
        this.courseName = courseName;
        this.baseFee = baseFee;
        this.level = level;
        this.hasCertificate = hasCertificate;
    }

    @Override
    public double getPaymentAmount() {
        if (baseFee < 0) {
            return 0;
        }

        double amount = baseFee;
        if (level == Level.INTERMEDIATE) {
            amount += 100;
        } else if (level == Level.ADVANCED) {
            amount += 200;
        }

        if (hasCertificate) {
            amount += 50;
        }
        return amount;
    }
}
```

Question 14 - KeywordCounter

```
import java.util.*;

interface Countable {
    int getKeywordCount();
}

class KeywordCounter implements Countable {
    private String text;
    private HashSet<String> keywords;
    private HashMap<String, Integer> wordCounts;

    public KeywordCounter(String text, Collection<String> keywords) {
        this.text = text;
        this.keywords = new HashSet<>(keywords);
```

```
        this.wordCounts = new HashMap<>();
        countWords();
    }

    private void countWords() {
        if (text == null || text.isEmpty()) {
            return;
        }

        String[] words = text.split(" ");
        for (String word : words) {
            if (wordCounts.containsKey(word)) {
                wordCounts.put(word, wordCounts.get(word) + 1);
            } else {
                wordCounts.put(word, 1);
            }
        }
    }

    @Override
    public int getKeywordCount() {
        if (text == null || text.isEmpty()) {
            return 0;
        }

        int total = 0;
        for (String keyword : keywords) {
            if (wordCounts.containsKey(keyword)) {
                total += wordCounts.get(keyword);
            }
        }
        return total;
    }
}
```